

第13章 模块系统:CJS、ESM 与包管理

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 C `#include` 的三个代价:文本复制、宏污染、链接期才报错
- 掌握 CJS 模块:`require` 的同步语义与模块缓存机制
- 掌握 ESM 模块:`import/export` 的静态语义、顶层 `await` 与 `import.meta`
- 掌握 `__dirname` 在两种模块下的差异与 `import.meta.url` 方案
- 掌握 CJS/ESM 互操作:`createRequire`、默认导出 `interop`、双包危害
- 掌握 `package.json` 的 `main/module/exports/type` 四个字段的职责
- 掌握 npm 工作流:`install`、`package-lock.json`、语义化版本

💡 从 C 到 JavaScript:本章主题如何迁移

C 里"把代码拼起来"靠预处理: `#include` 把头文件内容逐字复制进源文件。这套机制有众所周知的痛点:每个编译单元都背着整份头文件、宏污染命名空间、重复包含要靠 `#pragma once` 手动防御,而"符号找不到"要拖到链接期才报错。JavaScript 的模块系统走完了完整的三段进化: `#include` 是文本复制 → CJS 的 `require` 是"运行一次并缓存" → ESM 的 `import` 是"静态链接"。

CJS(CommonJS)诞生于 Node.js 早期:一个文件就是一个模块, `require()` 同步读文件、执行、把导出对象缓存起来,第二次 `require` 直接命中缓存。ESM(ECMAScript Modules)是现代标准: `import` 语句在解析阶段就被静态分析,名字、依赖图全部确定,因此编辑器能给出提示、打包器能做 tree-shaking(摇树优化,把没用的导出删掉)。这就像 C 的"声明与定义分离"升级成了"编译器真的认识你的接口"。

对 C 程序员来说本章最值钱的知识是两件事:一是 `__dirname` 在 ESM 里不存在,必须用 `import.meta.url` 自己换算——这是从 CJS 迁移到 ESM 的第一个坑;二是 `package.json` 的 `type/main/exports` 字段决定了 .js 文件被解析成哪种模块——扩展名不是唯一的判据。第13章的实验是一个"数学库 + 命令行工具"的小项目,你会同时用上 ESM 导出、CJS 兼容层和 npm 依赖,完成从"写一个 .c 文件"到"组织一个 Node 包"的完整过渡。

📦 前置知识自查

- C 预处理:`#include`、`#pragma once`、头文件与源文件分离
- 第2章解构赋值(`import` 语句中常见);第3章作用域与闭包
- 第4章函数一等公民;第7章 `process.argv` 与命令行参数

🚀 本章学习路线

1. **第一阶段(C 的基线)**:读 §13.1,回顾 `#include` 的三个代价
2. **第二阶段(CJS)**:读 §13.2,把 `require/module.exports`/缓存机制过一遍
3. **第三阶段(ESM)**:读 §13.3-§13.4,重点对比差异表与 `__dirname` 方案
4. **第四阶段(工程化)**:读 §13.5-§13.7,理解互操作、`package.json` 与 npm
5. **验收标准**:能独立搭出一个"ESM 库 + CLI 入口 + CJS 兼容"的小包,并解释每个字段的作用

本章知识导图

- 第13章 模块系统:CJS、ESM 与包管理
 - §13.1 C 的 #include:文本复制与它的代价
 - §13.2 CJS:require 与 module.exports
 - §13.3 ESM:import 与 export
 - §13.4 ESM 与 CJS 的关键差异
 - §13.5 互操作陷阱:createRequire 与双包危害
 - §13.6 package.json:模块的身份证
 - §13.7 npm:依赖管理与版本语义

§13.1 C 的 #include:文本复制与它的代价

```
// math.h — 头文件:声明 + 宏,等待被"复制"
#pragma once           // 防重复包含:同一文件只展开一次
int add(int a, int b);
#define SQUARE(x) ((x) * (x)) // 宏:纯文本替换,无类型检查

// main.c — 预处理阶段,math.h 的内容被逐字复制进本文件
#include "math.h"
#include <stdio.h>

int main(void) {
    printf("%d\n", add(2, 3));
    printf("%d\n", SQUARE(3 + 1)); // 展开为 (3+1)*(3+1),结果是 16
    return 0;
}
```

C 的拼接发生在三个不同的阶段,这是它的核心特征:

- **预处理期(文本复制):**#include 只是"把文件内容贴进来"。每个 .c 编译单元各背一份头文件,头文件越大编译越慢;不写 `#pragma once` 就可能重复定义。
- **编译期(翻译单元内):**宏在此时展开。 `SQUARE(x)` 没有类型、没有作用域,还容易出括号事故—— `SQUARE(3+1)` 展开成 `(3+1)*(3+1)`。
- **链接期(符号拼接):**函数声明可以编译,但"函数到底在哪"要到链接期才知道。 `undefined reference` 是链接错误,不是编译错误——错误发现得越晚,修起来越贵。

⚠ 常见误区 13.1:认为 #include 是"接口"而不是"文本"

头文件里任何一行宏、typedef 都可能污染所有包含它的文件,且包含顺序敏感:A 头里定义了 B 头依赖的宏,B 才能展开成功。这就是为什么 C 项目有"头文件洁癖"文化。JS 的模块没有这个问题:每个模块是独立作用域,导入的名字必须显式声明,顺序无关。

§13.2 CJS:require 与 module.exports

```
// lib/math.js — CJS 模块:一个文件就是一个模块,导出对象决定"外可见面"
'use strict';

function add(a, b) { return a + b; }
function mul(a, b) { return a * b; }
const VERSION = '1.0.0';

// 方式一:整体赋值 module.exports(最常用,一个模块导出一个"包")
module.exports = { add, mul, VERSION };

// 方式二:给 exports 挂属性 — 等价,因为 exports 一开始就是
// module.exports 的引用;但注意:不能对 exports 整体重赋值!
// exports.add = add; 这样写也有效
// exports = { add }; 这样写无效(断开了引用,外部拿到的还是老对象)
```

```
// main.cjs — require 的语义:同步读取 + 执行一次 + 结果缓存
const math = require('./lib/math.js');

console.log(math.add(2, 3));      // 5
console.log(math.VERSION);       // '1.0.0'

// 缓存:第二次 require 不会重新执行模块代码
const again = require('./lib/math.js');
console.log(again === math);     // true — 同一个对象,模块只执行一次

// require.cache 里能看到所有已加载模块(生产调试偶尔有用)
console.log(Object.keys(require.cache).length >= 1); // true
```

把 CJS 与 C 对照看: `#include` 是"复制", `require` 是"执行 + 缓存"。

- **加载即执行**: `require` 同步地读取文件、执行模块顶层代码、返回 `module.exports`。模块顶层代码只会执行一次(首次 `require`),之后都命中缓存。
- **作用域隔离**: 模块内声明的变量默认不对外可见,只有挂到 `module.exports` 上的才可见——比头文件干净得多,没有宏污染问题。
- **__dirname/ __filename**: CJS 模块里直接可用,指向当前文件目录与完整路径(对应 C 里"程序自己在哪"——C 没有标准答案,只有 `argv[0]` 的近似)。
- **循环依赖**: `require` 遇到循环时会返回"对方当前已完成的部分导出"——这也是为什么生产建议"尽量让依赖图无环"。

⚠ 常见误区 13.2:exports 整体重赋值

`exports = { add }` 不会生效: `exports` 只是 `module.exports` 的别名,重赋值只是把局部变量换个指向, `module.exports` 还是原来的空对象。规则: 要么一直用 `module.exports`, 要么一直用 `exports.xxx`, 不要混着用,更不要整体重赋值。

§13.3 ESM:import 与 export

```
// lib/math.mjs — ESM 模块:导出声明(语法级别,静态可分析)
export function add(a, b) { return a + b; }
export function mul(a, b) { return a * b; }
export const VERSION = '1.0.0';           // 命名导出:可以导出任何绑定
export default { add, mul };              // 默认导出:一个模块至多一个
```

```
// main.mjs — 导入端
import math, { add, mul, VERSION } from './lib/math.mjs';
//      ↑默认导出      ↑命名导出(名字必须精确匹配)

console.log(add(1, 2));           // 3
console.log(math.mul(3, 4));      // 12 — 默认导出是一个对象

// 重命名:避免命名冲突
import { VERSION as VER } from './lib/math.mjs';
console.log(VER);                 // '1.0.0'
```

ESM 与 CJS 的本质差异是 **静态分析**:import/export 必须出现在模块顶层(不能写在 if 里),依赖关系在解析阶段就完全确定。这带来三个工程红利:

- **编辑器提示**:导入导出名字错了立刻标红,像 C 的“编译期检查”提前到了写代码时。
- **tree-shaking**:打包器可以精确删除没被 import 的导出(比如只用了 add,没用的 mul 不进 bundle)。
- **顶层 await**:ESM 允许在模块顶层写 await(加载依赖时等一等),CJS 不行。

```
// config.mjs — 顶层 await:模块加载阶段异步初始化
const resp = await fetch('https://example.com/api/config');
export const config = await resp.json();
// 别的模块 import config 时,已经拿到了完整配置
```

⚠ 常见误区 13.3:import 里想用变量拼路径

import path from './' + name + '.mjs' 是语法错误——import 必须静态可分析,路径不能动态拼接。动态加载要用 import() 函数(见 §13.5),它返回 Promise。

§13.4 ESM 与 CJS 的关键差异

差异点	CJS	ESM
加载方式	require 同步执行	import 异步加载(顶层 await 可用)
__dirname/ __filename	直接可用	不存在,用 import.meta.url 换算
顶层 this	module.exports	undefined
严格模式	非默认(需 'use strict')	默认严格
语法位置	require 可写在任意位置	import 必须顶层;动态用 import()
扩展名判据	.cjs 或 type=commonjs	.mjs 或 type=module

最大的迁移坑是 `__dirname` 的消失:

```
// main.mjs — ESM 里没有 __dirname,需要自己从 import.meta.url 换算
import { fileURLToPath } from 'node:url';
import path from 'node:path';

console.log(import.meta.url);
// file:///C:/app/main.mjs — 这是"模块自身的 URL"

const __dirname = path.dirname(fileURLToPath(import.meta.url));
console.log(__dirname);
// C:\app — 与 CJS 的 __dirname 对齐

// 读同目录文件的老姿势在 ESM 里会踩坑:
// fs.readFileSync('./data.txt') 是相对"进程工作目录"的,不是模块目录
// 正确姿势:path.join(__dirname, 'data.txt')
```

⚠ 常见误区 13.4:ESM 里直接写 `__dirname`

在 `.mjs`(或 `type=module` 的 `.js`)里写 `__dirname` 会直接 `ReferenceError` ——它不是关键字,只是 CJS 注入的变量。生产包常用一个小工具函数封装换算逻辑,或直接用 `import.meta.dirname` (Node 20.11+ 提供,等价封装)。

§13.5 互操作陷阱:createRequire 与双包危害

```
// tool.mjs — ESM 里要用 require?用 createRequire 造一个
import { createRequire } from 'node:module';

const require = createRequire(import.meta.url);
// 以"当前模块"为基准的 require:内建模块、CJS 包都能加载

const path = require('node:path');
const fs = require('node:fs');
console.log(fs.readFileSync('data.txt', 'utf8'));
```

互操作三条铁律(生产面试高频):

- **ESM import CJS:**可以。CJS 的 `module.exports` 会整体作为 default 导出;命名导出由静态分析尽力推断(同步赋值才猜得到),猜不到时只能 `import pkg from 'pkg'` 再解构——所以 CJS 包的文档往往建议 `const { f } = require('pkg')` 风格。
- **CJS require ESM:**不行。require 是同步的,而 ESM 的加载是异步的,require 会直接抛 `ERR_REQUIRE_ESM`。替代:在 CJS 里用动态 `import('pkg.mjs')` (返回 Promise)。
- **双包危害:**同一个包被 CJS 与 ESM 各加载一份,内存里出现两份模块实例——两份状态、两套类, `instanceof` 全部失效。根治靠 `package.json` 的 `exports` 条件导出(§13.6),让包只以一种模块形式暴露。

```
// legacy.cjs — 模拟一个"老旧第三方 CJS 依赖"
module.exports = { greet: () => 'legacy hello' };

// main.mjs — CJS 里没有 require 时,用动态 import 加载 ESM;反向同理
const legacy = await import('./legacy.cjs');
console.log(legacy.default.greet()); // 'legacy hello'
// 注意:跨模块类型拿 default — CJS 的 module.exports 整体是 default
```

§13.6 package.json:模块的身份证

```
{
  "name": "calc-tools",
  "version": "1.4.2",
  "description": "命令行计算器工具包",
  "type": "module",
  "main": "./bin/cli.js",
  "exports": {
    ".": "./lib/index.js",
    "./math": "./lib/math.js",
    "./package.json": "./package.json"
  },
  "bin": { "calc": "./bin/cli.js" },
  "scripts": { "start": "node bin/cli.js" },
  "dependencies": { "commander": "^12.0.0" },
  "devDependencies": { "eslint": "^9.0.0" }
}
```

字段	作用	说明
type	决定 .js 按哪种模块解析	"module" = ESM;"commonjs"(默认) = CJS;.mjs/.cjs 扩展名强制覆盖
main	老式入口(require 与老工具)	向后兼容字段;新包以 exports 为准
exports	现代入口 + 子路径白名单	"." 是主入口,其余是允许外部 import 的子路径;未列出的子路径一律不可导入
bin	命令行工具注册名	npm install 后,注册名直接可用(如 calc)
scripts	任务脚本	npm start / npm test 等,可复用本机 PATH

exports 还能做条件导出,一举解决双包危害——同一个包,让 CJS 消费者拿到 CJS 副本、ESM 消费者拿到 ESM 副本:

```
{
  "exports": {
    ".": {
      "import": "./lib/index.mjs", // ESM 消费者走这里
      "require": "./lib/index.cjs" // CJS 消费者走这里
    }
  }
}
```

§13.7 npm:依赖管理与版本语义

```
npm init -y          # 生成 package.json(对话式初始化)
npm install lodash    # 安装并写入 dependencies(运行时依赖)
npm install -D eslint # -D = 写入 devDependencies(仅开发期依赖)
npm ci               # 严格按 package-lock.json 安装,CI 环境必用
npm outdated         # 查看哪些依赖有新版
```

- **node_modules**: 依赖实际落盘目录, 体积巨大, 永不提交版本库, 由 package.json + lock 文件重建。
- **package-lock.json**: 把"版本范围"钉死为"确切版本 + 完整依赖树"。有了它, 任何人在任何机器 `npm ci` 得到完全相同的依赖——这是"我这边能跑"问题的标准解药。
- **语义化版本(SemVer)**: 主.次.补丁。 `^1.4.2` 允许 1.x 内更新(不跨主版本); `~1.4.2` 只允许补丁更新; `1.4.2` 精确锁定。主版本 0 意味着不稳定。
- **打包器一句话**: webpack/vite/rollup 把多个 ESM 源文件(含 node_modules 里的依赖)分析、摇树、合并成浏览器可直接运行的单个 bundle 文件——浏览器本身没有 Node 的模块加载器, 打包器就是"预链接器"。

⚠ 常见误区 13.5: 不提交 package-lock.json

锁文件不提交, 团队里就会出现"我这边能跑、你那边报错"——因为 `^1.4.2` 允许各自装到不同版本。规则: 应用项目必须提交 lock 文件并用 `npm ci`; 库项目才可以选择性忽略。

本章速查表

主题	速查要点
C #include	预处理期文本复制; 顺序敏感; 宏无类型; 错误拖到链接期
require	CJS 同步加载; 模块顶层代码只执行一次; 结果缓存(require.cache)
module.exports	CJS 导出对象; exports 是它的别名, 整体重赋值会断开
__dirname(CJS)	当前文件目录, 直接可用
import/export	ESM 静态分析; 必须顶层; 命名导出 + 至多一个默认导出
import.meta.url	ESM 模块自身 URL; 配合 fileURLToPath 还原 __dirname
顶层 await	仅 ESM; CJS 不支持
createRequire	ESM 里造 require: createRequire(import.meta.url)
ESM 导入 CJS	default = module.exports; 命名导出靠静态推断, 不可靠
CJS require ESM	不允许(同步 vs 异步); 改用动态 import()
双包危害	同一包被两种模块各加载一份; exports 条件导出根治
扩展名/type	.cjs/.mjs 强制; js 看 type 字段(module/commonjs)
exports 字段	现代入口 + 子路径白名单 + import/require 条件分支
语义化版本	^1.4.2 主版本内浮动; ~1.4.2 仅补丁; lock 文件钉死
npm ci	严格按 lock 安装; CI 必用; 应用项目必须提交 lock

最小必会清单

- 能说出 `#include`、`require`、`import` 三者的本质差异(复制/执行缓存/静态链接)
- 能解释 `require` 的缓存语义与循环依赖的部分导出行为
- 能解释 `exports` 整体重赋值为什么无效
- 能说出 ESM 的三个静态分析红利(提示/tree-shaking/顶层 `await`)
- 能在 ESM 里还原 `__dirname(fileURLToPath + import.meta.url)`
- 能说出 CJS/ESM 互操作的三个规则与双包危害
- 能解释 `type/main/exports` 三个字段的职责与优先级
- 能写出带条件导出的 `exports` 配置解决双包危害
- 能解释 `^` 与 `~` 的区别、`lock` 文件的作用、`npm ci` 的适用场景
- 能一句话解释打包器在浏览器场景下的角色

自测题

Q 1. C 的 `#include` 与 Node 的 `require` 本质区别是什么?

✓ 参考答案

`#include` 是预处理期的文本复制(每次编译各自展开,无缓存);`require` 是运行期的"执行一次 + 缓存结果",模块代码只跑一遍,第二次 `require` 直接返回缓存对象。

Q 2. 为什么 `exports = { add }` 无效,而 `module.exports = { add }` 有效?

✓ 参考答案

`exports` 只是 `module.exports` 的引用别名;整体重赋值只是让局部变量指向新对象,`module.exports` 本身不变,外部拿到的仍是原对象。必须直接操作 `module.exports`。

Q 3. 在 `.mjs` 文件里写 `__dirname` 会发生什么?正确做法是什么?

✓ 参考答案

`ReferenceError: __dirname 未定义`——它只是 CJS 注入的变量,不是语言关键字。正确做法:`fileURLToPath(import.meta.url) + path.dirname` 换算,或使用 `import.meta.dirname(Node 20.11+)`。

Q 4. 一个 CJS 包被"ESM 消费者"和"CJS 消费者"同时加载,会发生什么?如何根治?

✓ 参考答案

双包危害:两份模块实例、两套状态与类,`instanceof` 失效。根治:`package.json exports` 配置条件导出(`import/require` 分支),让每个消费者只拿到一种形态。

Q 5. `package.json` 里 `"type": "module"` 时,一个 `.cjs` 文件按哪种模块解析?为什么?

✅ 参考答案

按 CJS。扩展名优先于 type 字段: .cjs 强制 CJS、.mjs 强制 ESM, type 字段只决定"无扩展名标记的 .js"的解析方式。

Q 6. 团队里出现"我这边能跑,你那边报错",最可能的原因与标准解法是什么?

✅ 参考答案

依赖版本漂移: ^ 允许范围浮动,各人装到不同版本。标准解法:提交 package-lock.json,统一用 npm ci 按锁文件精确安装。

章末实验题:数学库 + CLI 工具包

📄 实验 13:数学库 + CLI 工具包(ESM/CJS 双模)

实验目标:搭建一个"库 + 命令行"双入口小包,覆盖 ESM 导出、CJS 兼容、package.json 配置与互操作。

实验要求:

- 任务 1(覆盖:ESM 命名导出):lib/math.js 导出 sum/avg/max/min 四个纯函数,支持任意个数的数值参数
- 任务 2(覆盖:CLI 入口、process.argv):bin/cli.js 支持 `node cli.js sum 1 2 3` 与 `node cli.js avg 4 5 6`, 参数不足时打印用法并退出码非 0
- 任务 3(覆盖:createRequire/互操作):cli.js 里用 createRequire 加载一个 CJS 兼容层 legacy.cjs(module.exports 导出版本号),并在输出中展示
- 任务 4(覆盖:package.json):配置 type=module、exports(".")与"./math"子路径)、bin 与 scripts
- 任务 5(覆盖:缓存语义):注释说明"为什么 cli.js 里 import 两次 math.js 只执行一次"
- 任务 6(覆盖:语义化版本):dependencies 里用 ^ 版本装一个内建无关的第三方包,注释说明版本范围语义(不必真实安装)

实验环境:Node.js 20+,运行 `node bin/cli.js sum 1 2 3`。

第一步 写纯函数库:lib/math.js 只做导出,不做任何 I/O——库与入口分离是模块设计第一课

第二步 写 CLI:解析 process.argv.slice(2),按子命令分发,统一处理"参数不是数字"的错误

第三步 后接互操作与配置:补 legacy.cjs 与 package.json,验证两种加载路径都能工作

✅ 参考答案要点

核心:纯函数库 + 入口分发 + createRequire 互操作 + package.json 双入口。约 120 行,覆盖全章模块知识点。

```
// lib/math.js — ESM 数学库(纯函数,无状态、无 I/O)
export function sum(...nums) {
  return nums.reduce((acc, n) => acc + n, 0);
}
export function avg(...nums) {
  if (nums.length === 0) throw new Error('avg: 参数不能为空');
  return sum(...nums) / nums.length;
}
export function max(...nums) {
  if (nums.length === 0) throw new Error('max: 参数不能为空');
  return Math.max(...nums);
}
export function min(...nums) {
  if (nums.length === 0) throw new Error('min: 参数不能为空');
  return Math.min(...nums);
}
```

```
// legacy.cjs — CJS 兼容层(模拟"老旧第三方依赖")
// 任务 3:ESM 里用 createRequire 加载它,验证 CJS 的导出形态
module.exports = {
  libName: 'calc-tools',
  version: '1.0.0',
  loadedBy: 'createRequire(cjs)',
};
```

```

// bin/cli.js — CLI 入口(ESM)
import { createRequire } from 'node:module';
import { sum, avg, max, min } from '../lib/math.js';

const require = createRequire(import.meta.url);
const legacy = require('../legacy.cjs'); // 互操作:CJS 模块整体是 default

function usage() {
  console.log('用法: node cli.js <sum|avg|max|min> 数字...');
  console.log('示例: node cli.js sum 1 2 3 → 6');
  process.exitCode = 1;
}

function parseNums(list) {
  const nums = list.map(Number);
  if (nums.some((n) => Number.isNaN(n))) {
    throw new Error('存在无法解析的数字: ' + list.join(' '));
  }
  return nums;
}

function main() {
  console.log('== ' + legacy.libName + ' v' + legacy.version + ' ==');

  const [cmd, ...rest] = process.argv.slice(2);
  if (!cmd || rest.length === 0) return usage();

  try {
    const nums = parseNums(rest);
    switch (cmd) {
      case 'sum': console.log('sum =', sum(...nums)); break;
      case 'avg': console.log('avg =', avg(...nums)); break;
      case 'max': console.log('max =', max(...nums)); break;
      case 'min': console.log('min =', min(...nums)); break;
      default: usage();
    }
  } catch (err) {
    console.error('错误: ' + err.message);
    process.exitCode = 1;
  }
}

// 任务 5:下面这行执行后,math.js 的顶层代码只跑一次(缓存语义);
// 再 import 一次拿到的还是同一份导出,不会重复执行模块代码。
main();

```

```
// package.json — 任务 4/6:双入口 + 子路径 + 版本语义
{
  "name": "calc-tools",
  "version": "1.0.0",
  "type": "module",
  "main": "./bin/cli.js",
  "exports": {
    ".": "./lib/math.js",
    "./math": "./lib/math.js"
  },
  "bin": { "calc": "./bin/cli.js" },
  "scripts": { "start": "node bin/cli.js sum 1 2 3" },
  "dependencies": { "commander": "^12.0.0" }
}
// ^12.0.0:主版本 12 内可浮动更新(12.x);~12.0.0 则只允许补丁(12.0.x)
```

设计说明:sum 用 rest 参数收集任意个数;parseNums 把 argv 里的字符串统一转数字并显式报错——模拟真实 CLI 的输入校验;legacy.cjs 演示 CJS 与 ESM 双模共存的场景,呼应 §13.5 互操作规则。运行 `node bin/cli.js sum 1 2 3` 输出 6;运行 `node bin/cli.js` 打印用法并以退出码 1 结束。

下一章预告:第14章 类型系统进阶 —— 结构类型与鸭子类型、JSON 边界、深浅拷贝、空安全,以及 TypeScript 与 JSDoc 渐进类型的方向。